МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ» ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №1

з дисципліни "Програмування систем на кристалі"

Студента 4 курсу
Спеціальності 123 –
"Комп'ютерна інженерія"
Гуцула Дениса Ігоровича

м. Ужгород – 2026 р.

Лабораторна робота №1

Тема: Налаштування системи тактування (Clocks) та обробники переривань (ISR) у PSoC 4

Мета: Ознайомитися з тактуванням та обробкою переривань (ISR) для мікроконтролерів PSoC 4, використовуючи засоби інтегрованого середовища PSoC® Creator™, а також підключати й використовувати логічний аналізатор Saleae Logic для апаратної верифікації розробленого проєкту.

Хід виконання завдання

Завдання 1 Розрахунки тактування, основних параметрів і похибок.

Перед тим як складати схему та виконувати подальші модифікації проєкту, треба визначитись з тактуванням, розрахувати дільник джерела, фактичну частоту, відносну похибку частоти шини, дільник для локального таймера, його фактичну частоту й період перемикання сигналу основного циклу. Вхідні параметри (варіант 5) надано в таблиці 1.

Таблиця 1 – Варіант завдання

Варіант	IMO, МГц	Бажаний BUS CLK	Тік Timer (Clock_1)	Період переривання Timer_1	CyDelay в main()
5	48	24 МГц	4 МГц (0.25 мкс)	100 мкс	1 мс

Розрахунок дільника джерела BUS_Clk:

Теоретично, для того щоб отримати 24 МГц, дільник має бути 2, так як IMO = 48 МГц.

$$divider_bus = IMO / BUS_CLK_desired = 48\ МГц / 24\ МГц = 2$$

Фактична частота BUS_Clk (Nominal/Actual):

Фактичну частоту джерела можна знайти, якщо поділити частоту базового генератора на вирахуваний раніше дільник.

$$BUS_CLK = IMO / divider_bus = 48\ МГц / 2 = 24\ МГц$$

У мікроконтролері CY8C4245AXI-483 (PSoC 4200) роль BUS CLK виконує SysClk: HFCLK = IMO = 48 МГц, а SysClk = HFCLK / 2 = 24 МГц. Саме це значення (Divider = 2, Nominal 24 MHz) показує редактор Clocks у lab1.cydwr.

Відносна похибка частоти шини:

Для обрахунку відносної похибки частоти шини використовується наступний вираз:

$$\delta = (BUS_CLK_actual - BUS_CLK_desired) / BUS_CLK_desired \cdot 100\ \% = (24 - 24) / 24 \cdot 100\ \% = 0\ \%$$

Розрахунок дільника для локального клоку таймера (Clock_1):

Для формування частоти 4 МГц джерело BUS_CLK (24 МГц) ділиться середовищем автоматично:

$$divider_Clock_1 = BUS_CLK / Clock_1_desired = 24\ МГц / 4\ МГц = 6$$

Оскільки в PSoC 4200 периферійні дільники тактуються безпосередньо від HFCLK (48 МГц), PSoC Creator підібрав для Clock_1 дільник 48 МГц / 4 МГц = 12. В обох випадках дільник цілий, тому фактична частота Clock_1: 4 МГц (похибка 0 %), тік таймера 1 / 4 МГц = 0.25 мкс.

Розрахунок значення регістру періоду таймера (Timer_1 Period):

Період таймера розраховується за наступною формулою:

$$Period = T_ISR / T_tick = 100\ мкс / 0.25\ мкс = 400\ тіків$$

Лічильник TCPWM у режимі Timer/Counter рахує від 0 до значення Period включно, тобто подія Terminal Count настає кожні (Period + 1) тіків.

Тому в регістр Period записано 400 − 1 = 399 (16-бітний лічильник, максимум 65535 – вистачає).

Розрахунок періоду перемикання сигналу основного циклу (Pin_Main):

Так як в основному циклі виконується послідовність: Pin_Main_Write(1), CyDelay(1 ms), Pin_Main_Write(0), CyDelay(1 ms), то загальний розрахунковий період прямокутного сигналу на Pin_Main: T_Main = 1 мс + 1 мс = 2 мс.

Завдання 2 Побудувати схему й налаштувати піни

Для того щоб виконати завдання, потрібно побудувати схему з компонентів, наведених у таблиці 2.

Таблиця 2 – Компоненти схеми

Компонент	Назва	Налаштування
Clock	Clock_1	Джерело – HFCLK (BUS CLK), частота 4 МГц, дільник 12
TCPWM	Timer_1	Вхідний клок – Clock_1; Mode = Timer/Counter; Period = 399; переривання по Terminal Count
Interrupt (isr)	isr_Timer	Вхід підключити до виходу переривання Timer_1
Digital Output Pin	Pin_ISR	Порт P0_2 (виведений на RGB-світлодіод на платі), Strong Drive
Digital Output Pin	Pin_Main	Порт P0_3 (виведений на RGB-світлодіод на платі), Strong Drive

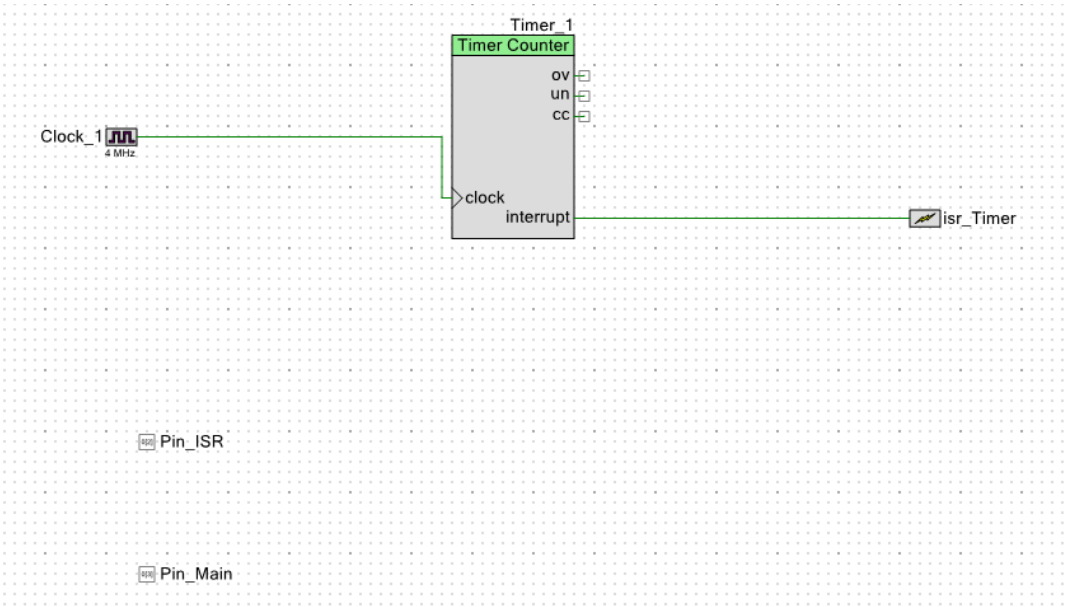


Рисунок 1 – Побудована схема

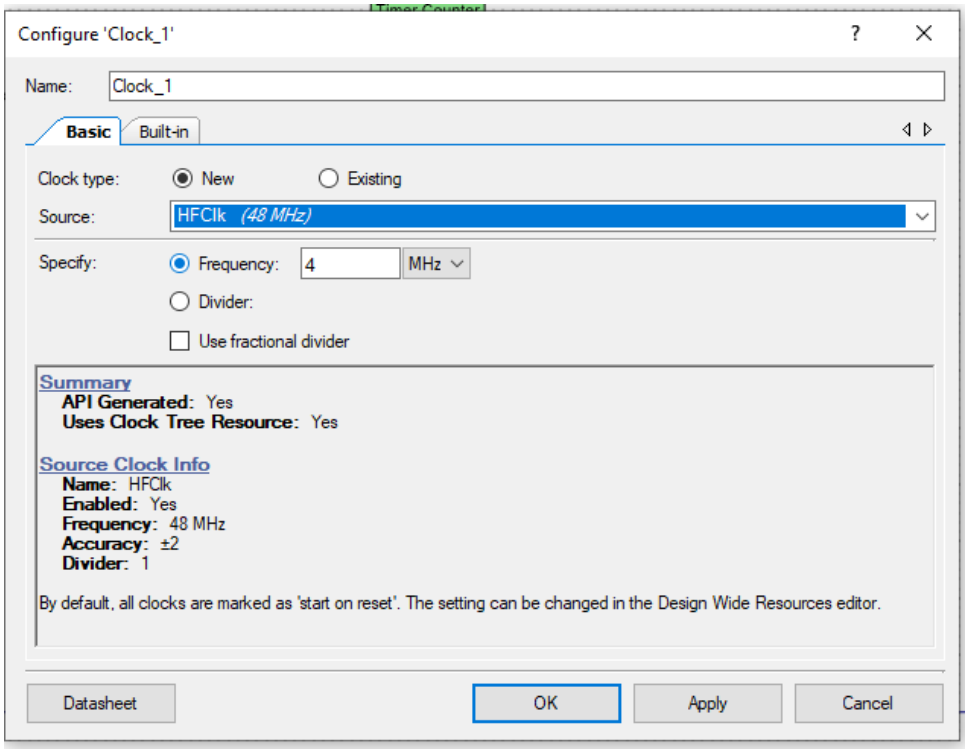


Рисунок 2 – Налаштування Clock

Також потрібно налаштувати піни. Порти для пінів зазначені в таблиці 2, а налаштування пінів зображено на рисунку 3 (пін Button використовується в завданні 4).

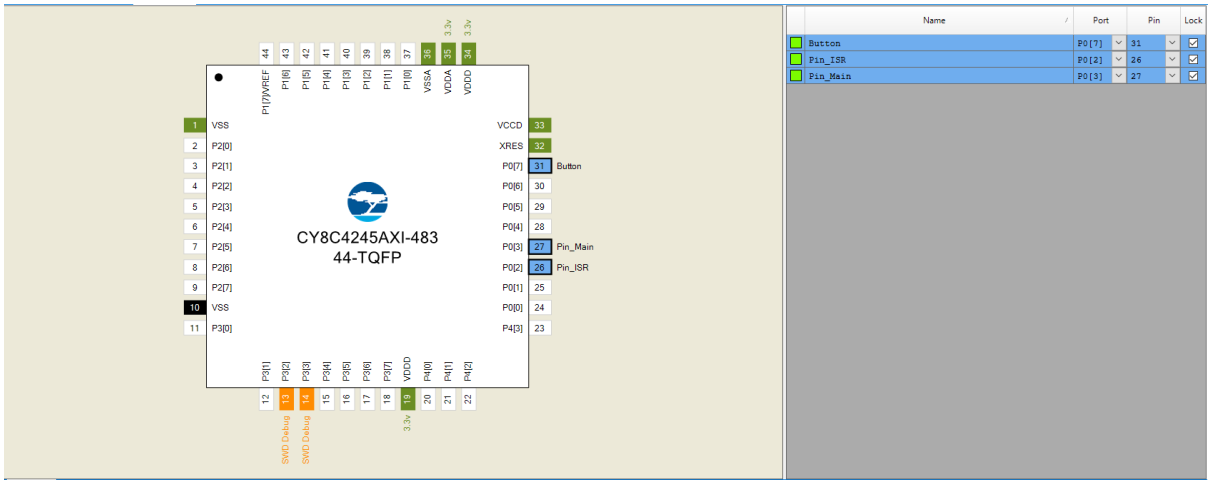


Рисунок 3 – Налаштовані піни

Тактування проєкту (вкладка Clocks редактора lab1.cydwr) наведено на рисунку 4: IMO = 48 MHz, SysClk = 24 MHz (Divider 2), Clock_1 = 4 MHz (Divider 12 від HFCLK).

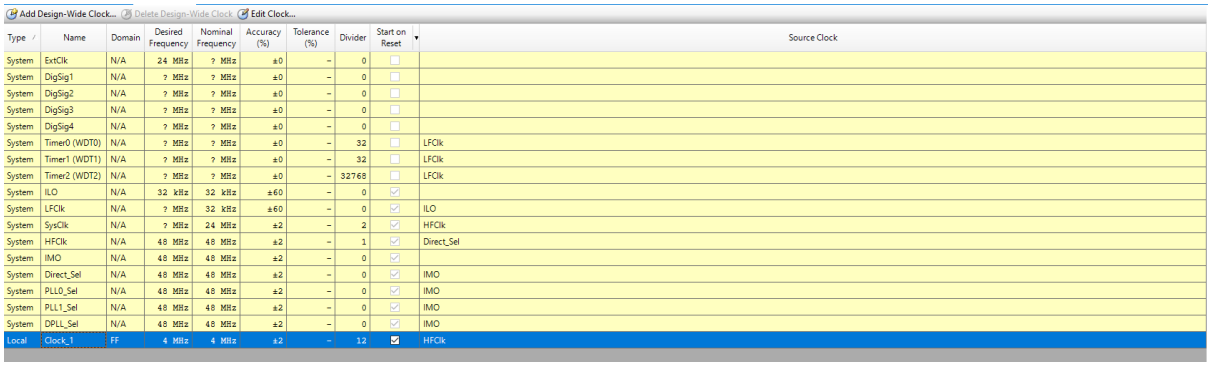


Рисунок 4 – Налаштування тактування в lab1.cydwr

Лістинг коду з main.c:

```
#include "project.h"

/* Варіант 5: Clock_1 = 4 МГц (тік 0.25 мкс), період переривання 100 мкс -> 400 тіків.
 * TCPWM рахує від 0 до Period включно, тому в регістр записується 400 - 1 = 399. */
#define TIMER_PERIOD_TICKS (400u - 1u)

volatile uint32_t isr_timer_count = 0u;

CY_ISR(Timer_ISR_Handler)
{
    Pin_ISR_Write((uint8)(!Pin_ISR_Read()));

    isr_timer_count++;
```

```
    Timer_1_ClearInterrupt(Timer_1_INTR_MASK_TC);
}

int main(void)
{
    CyGlobalIntEnable;

    Clock_1_Start();
    Timer_1_Start();
    Timer_1_WritePeriod(TIMER_PERIOD_TICKS);
    Timer_1_SetInterruptMode(Timer_1_INTR_MASK_TC);
    isr_Timer_StartEx(Timer_ISR_Handler);

    for (;;)
    {
        Pin_Main_Write(1u);
        CyDelay(1);
        Pin_Main_Write(0u);
        CyDelay(1);
    }
}
```

Проєкт зібрано без помилок (Build Succeeded), результат збірки показано на рисунку 5.



Рисунок 5 – Результат збірки проєкту

Завдання 3 Виконати вимірювання за допомогою Saleae Logic 2.0

Використовуючи додаток Logic 2.0, а також логічний аналізатор, виконуються вимірювання значень реального періоду переривання (Pin_ISR, P0_2), тривалості ISR (ширина імпульсу на Pin_ISR) та періоду сигналу основного циклу (Pin_Main, P0_3). Канал D7 аналізатора підключено до P0_2, канал D6 – до P0_3, GND – до землі плати. Результати вимірювань наведено на рисунках 6–8.

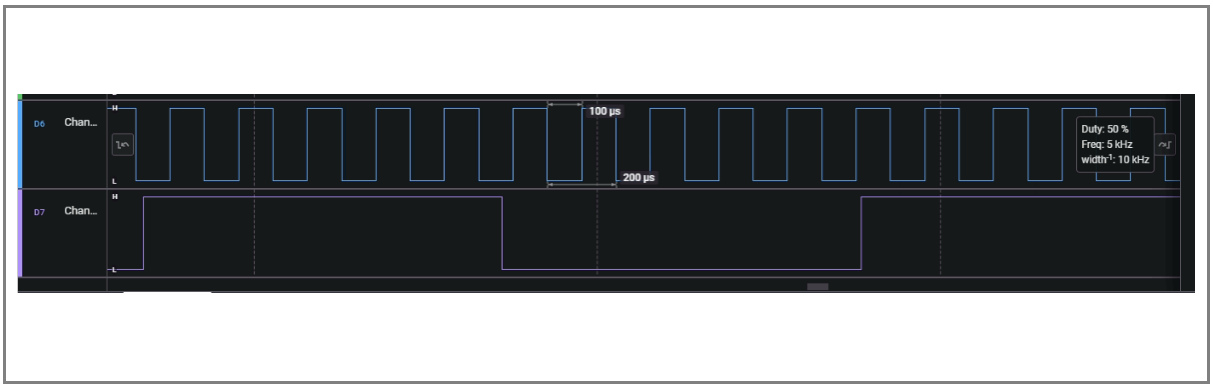


Рисунок 6 – Сигнали Pin_ISR (D6) і Pin_Main (D7) у Logic 2.0

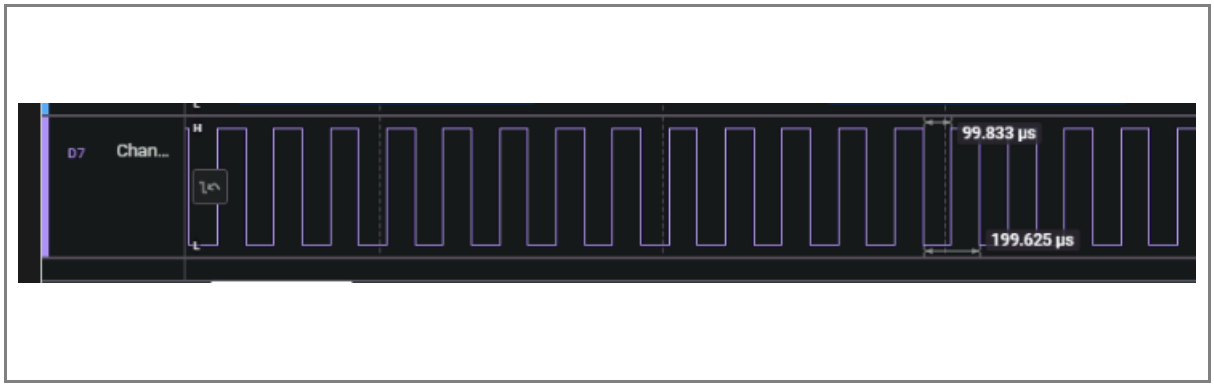


Рисунок 7 – Вимірювання періоду переривання на Pin_ISR (очікується 100 мкс)

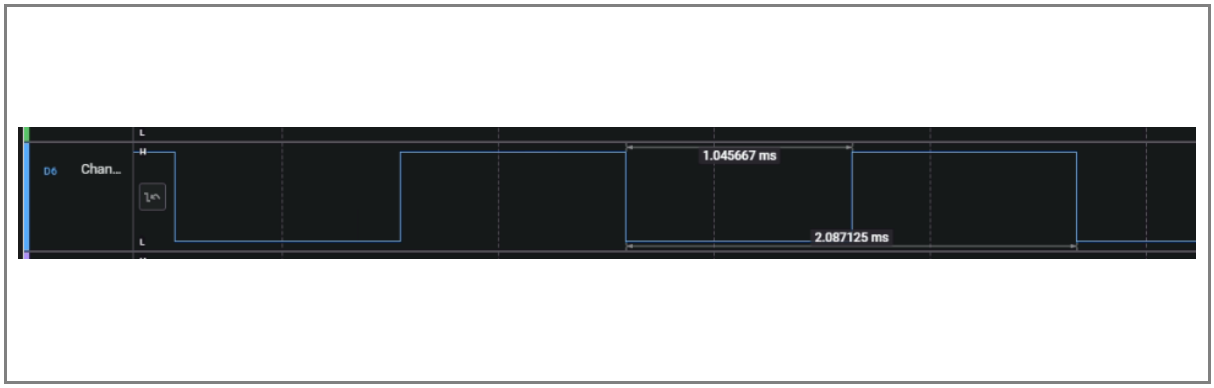


Рисунок 8 – Вимірювання періоду сигналу Pin_Main (очікується 2 мс)

Всі результати вимірювань і розрахунків наведено в таблиці 3.

Завдання 4 Модифікувати переривання так, щоб від фізичної кнопки вмикався/вимикався світлодіод

Для того щоб виконати завдання, було додано кнопку (Digital Input Pin Button, порт P0_7, Resistive pull up, переривання по спадаючому фронту – Falling edge) і переривання для неї (isr_Button, підключене до виходу irq піна), а також модифіковано код, додавши до нього обробник переривання фізичної кнопки. Кнопка SW2 плати CY8CKIT-042 при натисканні замикає P0_7 на землю, тому обробник спрацьовує один раз на кожне натискання й інвертує стан світлодіода Pin_Main (P0_3).

```
CY_ISR(Button_Main_Handler)
{
    uint8 current_state = Pin_Main_Read();
    Pin_Main_Write((uint8)(!current_state));

    Button_ClearInterrupt();
}
```

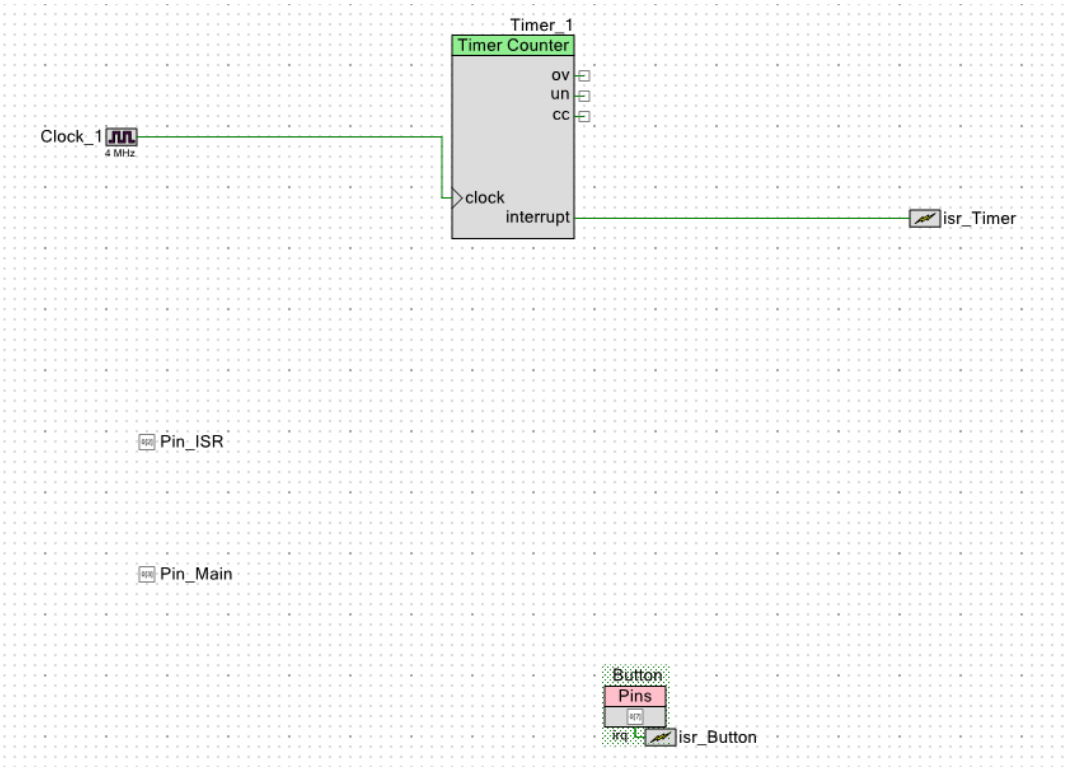


Рисунок 9 – Модифікована схема

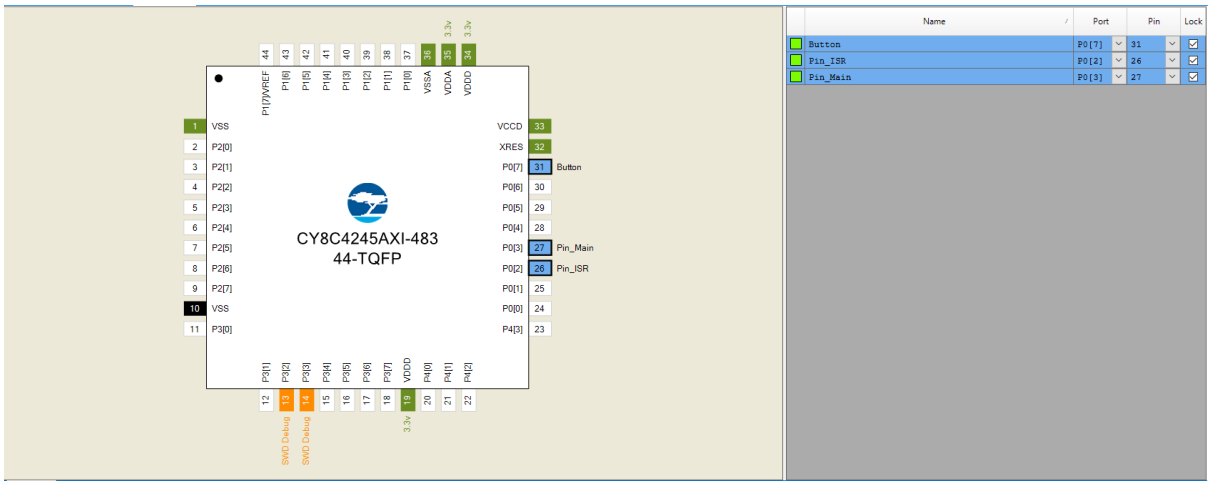


Рисунок 10 – Модифіковані порти

Лістинг модифікованого коду:

```
#include "project.h"
```

```
/* Варіант 5: Clock_1 = 4 МГц (тік 0.25 мкс), період переривання 100 мкс ->
400 тіків.
 * TCPWM рахує від 0 до Period включно, тому в регістр записується 400 - 1
= 399. */
#define TIMER_PERIOD_TICKS (400u - 1u)

volatile uint32_t isr_timer_count = 0u;

CY_ISR(Timer_ISR_Handler)
{
    Pin_ISR_Write((uint8)(!Pin_ISR_Read()));

    isr_timer_count++;

    Timer_1_ClearInterrupt(Timer_1_INTR_MASK_TC);
}

CY_ISR(Button_Main_Handler)
{
    uint8 current_state = Pin_Main_Read();
    Pin_Main_Write((uint8)(!current_state));

    Button_ClearInterrupt();
}

int main(void)
{
    CyGlobalIntEnable;

    Clock_1_Start();
    Timer_1_Start();
    Timer_1_WritePeriod(TIMER_PERIOD_TICKS);
    Timer_1_SetInterruptMode(Timer_1_INTR_MASK_TC);

    isr_Timer_StartEx(Timer_ISR_Handler);
    isr_Button_StartEx(Button_Main_Handler);

    for (;;)
    {
        /* Pin_Main перемикається лише в обробнику переривання кнопки */
    }
}

/* [] END OF FILE */
```

Роботу кнопки перевірено за допомогою Logic 2.0 з тим самим підключенням аналізатора (D7 – Pin_ISR, P0_2; D6 – Pin_Main, P0_3). Після прошивки модифікованого проєкту запускається запис і кілька разів натискається кнопка. Сигнал Pin_ISR і далі перемикається кожні 100 мкс, а рівень Pin_Main змінюється лише в момент натискання кнопки (рисунки 11–12).

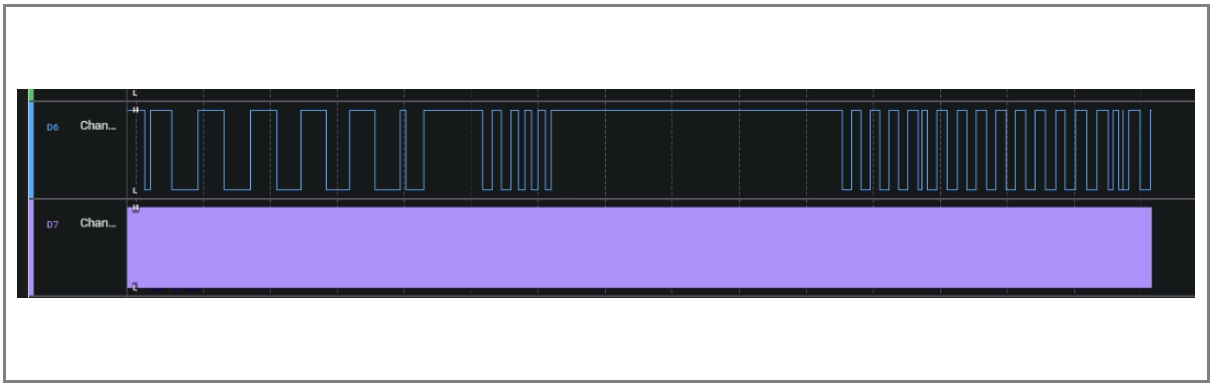


Рисунок 11 – Сигнали Pin_ISR (D7) і Pin_Main (D6) під час кількох натискань кнопки

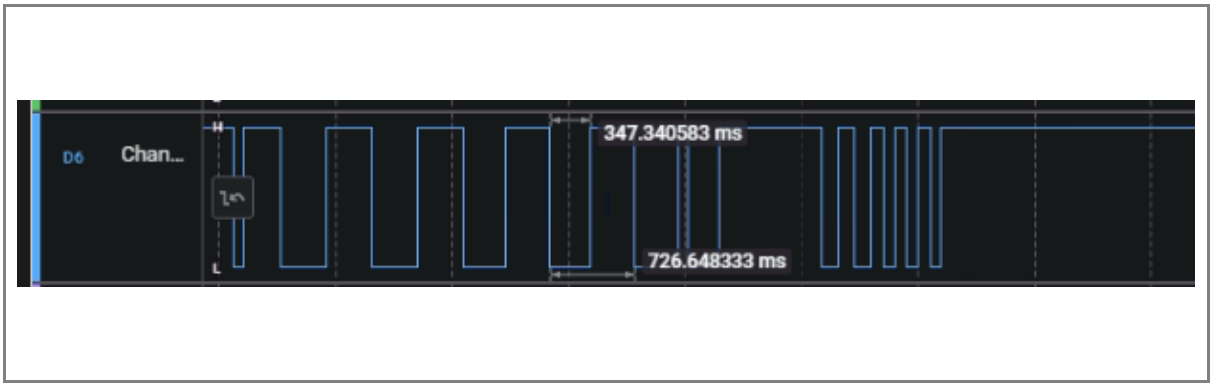


Рисунок 12 – Перемикання Pin_Main після натискання кнопки (збільшений масштаб)

Таблиця 3 – Результати розрахунків і вимірювань

Параметр / Сигнал	**Розрахункове**	**IDE Actual / Nominal**	**Виміряне (Saleae Logic 2)**
IMO	48 МГц	48 МГц	–
BUS_CLK (SysClk)	24 МГц	24 МГц (дільник = 2)	–
Clock_1	4 МГц	4 МГц (дільник = 12)	–
Регістр Period (Timer_1)	399 (400 тіків)	399	–
Період сигналу ISR (Pin_ISR)	0.100 мс	0.100 мс	0.0998 мс (99.833 мкс)
Період сигналу main (Pin_Main)	2.000 мс	2.000 мс	2.087 мс

Висновки: під час виконання даної лабораторної роботи було ознайомлено з тактуванням та обробником переривань (ISR) для мікроконтролерів PSoC 4, використовуючи засоби інтегрованого середовища PSoC® Creator™. Для варіанта 5 налаштовано IMO = 48 МГц, BUS_CLK = 24 МГц (дільник 2), Clock_1 = 4 МГц та Period = 399 (400 тіків по 0.25 мкс = 100 мкс); усі дільники цілі, тому розрахункові значення збігаються з фактичними значеннями IDE (похибка 0 %). Побудовано схему з таймером TCPWM, обробником переривання та вихідними пінами, проєкт успішно зібрано, а також додано переривання від фізичної кнопки P0.7, яке вмикає/вимикає світлодіод. Вимірювання логічним аналізатором Saleae Logic на платі CY8CKIT-042 показали період переривання 99.833 мкс (відхилення −0.17 %, у межах точності генератора IMO) і період сигналу основного циклу 2.087 мс (+4.4 %). Основний цикл довший за розрахунок, бо програмна затримка CyDelay() не враховує час, який процесор витрачає на обробку переривань кожні 100 мкс, тоді як апаратний таймер від них не залежить.